

Il caso di due studenti — fascicolo

Lab6 · Incontro 4 · blocco 4. Marco e Giulia sono **inventati**: le loro consegne sono state costruite per il laboratorio, con repository Git veri. I test sono stati eseguiti davvero; gli output qui sotto sono quelli reali.

La consegna (livello 4, AI-powered) — mini-progetto «Report di fine giornata»: a fine giornata la titolare vuole sapere quanti scontrini sono stati fatti, l'incasso totale (IVA 22% inclusa, sconto del 10% sugli scontrini che superano 50 € con IVA), lo scontrino più alto, quanti articoli sono costati più di 20 € (prezzo netto). Un reso è un prezzo negativo. Si consegnano: specifica controllata con l'allenatore di prompt; almeno 5 test scritti **prima** del codice; prompt log; repository con almeno 4 commit significativi; dichiarazione d'uso dell'IA. In classe: 5 minuti di orale con una modifica dal vivo, senza IA.

Il vostro compito (20 minuti, individuale).

1. Leggete le due consegne.
2. Compilate la rubrica per ciascuno (ultima pagina). Per ogni criterio scrivete **l'evidenza** su cui vi basate.
3. Scrivete **tre domande** per l'orale di ciascuno: devono servire a capire se lo studente ha capito il suo codice.

Marco

Storia dei commit (`git log`):

```
c4892c9  dom 15/03 23:58  progetto finito
```

SPECIFICA.md

```
# Specifica
```

Il programma calcola il report di fine giornata del negozio: numero di scontrini, incasso, scontrino più alto e articoli sopra 20 euro.

report.py

```
from typing import Dict, List

IVA = 0.22

def totale(scontrino: List[float]) -> float:
    lordo = sum(scontrino) * (1 + IVA)
    return round(lordo * 0.9 if lordo > 50 else lordo, 2)

def report(giornata: List[List[float]]) -> Dict[str, float]:
    totali = [totale(s) for s in giornata]
    return {
        "scontrini": len(giornata),
        "incasso": round(sum(totali), 2),
        "scontrino_piu_alto": max(totali),
        "articoli_sopra_20": sum(1 for s in giornata for p in s if p > 20),
    }
```

test_report.py

```
from report import report

def test_un_scontrino():
    assert report([[10, 20, 15]])["incasso"] == 49.41

def test_numero_scontrini():
    assert report([[10], [20], [30]])["scontrini"] == 3

def test_piu_alto():
    assert report([[10], [30, 20]])["scontrino_piu_alto"] == 54.9

def test_sopra_20():
    assert report([[10, 25], [30]])["articoli_sopra_20"] == 2

def test_due_scontrini():
    assert report([[10, 20, 15], [5]])["incasso"] == 55.51
```

Esito dei test (pytest -q):

```
..... [100%]
5 passed in 0.01s
```

Prompt log: non consegnato. **Dichiarazione d'uso dell'IA:** non consegnata.

Giulia

Storia dei commit (git log):

```
ffac71c  lun 09/03 16:10  Specifica del report: prima versione
1cb4c3f  lun 09/03 17:02  Specifica: casi limite dopo l'allenatore di prompt (resi, 20 esatti, 50
esatti, giornata vuota)
48e317e  mar 10/03 15:30  Test scritti prima del codice: 6 casi
dcc5fab  mer 11/03 18:45  Prima versione generata con l'IA (da rivedere)
65bdbb1  gio 12/03 16:20  Riscritto con for e if; corretto l'ordine: prima IVA, poi sconto
albbcac  ven 13/03 15:05  Test: aggiunti solo resi e tanti articoli piccoli
0869e24  sab 14/03 11:40  Prompt log: perché fallisce il test della giornata vuota
8b993b5  dom 15/03 10:15  README e dichiarazione d'uso dell'IA
adeb9d5  dom 15/03 18:30  README: segnalato il test rosso lasciato apposta
```

SPECIFICA.md

```
# Specifica del report di fine giornata

**Scopo.** A fine giornata la titolare vuole un riepilogo degli scontrini.

**Input.** La lista degli scontrini della giornata; ogni scontrino è una lista di prezzi netti
(senza IVA).

**Output.** Quattro valori: numero di scontrini, incasso totale, scontrino più alto, articoli sopra
20 €.

**Regole.** IVA 22% sui prezzi netti; se il totale di uno scontrino con IVA supera 50 €, sconto del
10% su quello scontrino; arrotondamento ai centesimi.

**Casi limite** (aggiunti dopo l'allenatore di prompt).
- Un reso è un prezzo negativo: si somma come gli altri.
- Un articolo da 20 € esatti non conta tra quelli «sopra 20 €».
- Uno scontrino con totale ivato di 50 € esatti non ha lo sconto («supera» vuol dire più di 50).
- Giornata senza scontrini: tutti i valori a 0. *(Da confermare con il professore.)*

**Come verifico.** Test in `test_report.py`, scritti prima del codice.
```

report.py

```
IVA = 0.22
SOGLIA_SCONTO = 50
SCONTO = 0.10

def totale_scontrino(prezzi):
    totale = 0
    for prezzo in prezzi:
        totale = totale + prezzo
    totale = totale * (1 + IVA)
    if totale > SOGLIA_SCONTO:
        totale = totale - totale * SCONTO
    return round(totale, 2)

def report_giornata(scontrini):
    incasso = 0
    piu_alto = None
    sopra_20 = 0
    for prezzi in scontrini:
        t = totale_scontrino(prezzi)
        incasso = incasso + t
        if piu_alto is None or t > piu_alto:
            piu_alto = t
        for prezzo in prezzi:
            if prezzo > 20:
                sopra_20 = sopra_20 + 1
    return len(scontrini), round(incasso, 2), piu_alto, sopra_20
```

test_report.py

```

from report import report_giornata

def test_un_solo_scontrino():
    # [10, 20, 15]: 45 netti, 54,90 con IVA, supera 50: sconto, 49,41
    assert report_giornata([[10, 20, 15]]) == (1, 49.41, 49.41, 0)

def test_sconto_su_secondo_scontrino():
    # [40]: 48,80 con IVA, niente sconto; [30, 20]: 61,00 con IVA, sconto: 54,90
    assert report_giornata([[40], [30, 20]]) == (2, 103.7, 54.9, 2)

def test_articolo_da_20_esatti_non_conta():
    assert report_giornata([[20, 21]])[3] == 1

def test_reso_negativo():
    # [30, -10]: 20 netti, 24,40 con IVA
    assert report_giornata([[30, -10]]) == (1, 24.4, 24.4, 1)

def test_piu_alto_tra_tre():
    assert report_giornata([[5], [10], [8]])[2] == 12.2

def test_giornata_vuota():
    assert report_giornata([]) == (0, 0, 0, 0)

def test_solo_resi():
    # [-10]: -12,20 con IVA, nessuno sconto
    assert report_giornata([[-10]]) == (1, -12.2, -12.2, 0)

def test_tanti_articoli_piccoli():
    # dieci articoli da 5: 50 netti, 61,00 con IVA, sconto: 54,90
    assert report_giornata([[5, 5, 5, 5, 5, 5, 5, 5, 5, 5]]) == (1, 54.9, 54.9, 0)

```

Esito dei test (`pytest -q`):

```

.....F.. [100%]
===== FAILURES =====
E   assert (0, 0, None, 0) == (0, 0, 0, 0)

        At index 2 diff: None != 0
        Use -v to get more diff
/tmp/tmpucv98il2/giulia/test_report.py:28: assert (0, 0, None, 0) == (0, 0, 0, 0)
===== short test summary info =====
FAILED test_report.py::test_giornata_vuota - assert (0, 0, None, 0) == (0, 0,...
1 failed, 7 passed in 0.01s

```

PROMPT_LOG.md

Prompt log

#	Data	Cosa ho chiesto	Cosa ho accettato	Cosa ho cambiato e perché
1	11/03	Ho incollato la mia specifica e i test e ho chiesto la funzione <code>report_giornata</code>	La struttura del ciclo sugli scontrini	Da rivedere: usa <code>sum</code> , <code>max</code> e una list comprehension che non abbiamo fatto
2	12/03	«Riscrivi senza <code>sum</code> , <code>max</code> e list comprehension, solo con <code>for</code> e <code>if</code> »	Il ciclo con l'accumulatore	Due test erano rossi: lo sconto era calcolato sul netto, prima dell'IVA (come lo studente C della lezione sulle consegne ambigue). L'ho corretto io: prima l'IVA, poi lo sconto
3	14/03	«Perché il mio test sulla giornata vuota fallisce?»	La spiegazione: con nessuno scontrino <code>piu_alto</code> resta <code>None</code>	Non ho cambiato il codice: nella specifica avevo scritto 0, ma non so se è giusto. Lo chiedo al professore. Il test resta rosso apposta

README.md (con la dichiarazione d'uso)

Report di fine giornata – Giulia

- ``SPECIFICA.md``: la specifica, controllata con l'allenatore di prompt.
- ``test_report.py``: i test, scritti prima del codice (vedi la storia dei commit).
- ``report.py``: il programma.
- ``PROMPT_LOG.md``: cosa ho chiesto all'IA e cosa ho cambiato.

△ Un test fallisce di proposito (giornata vuota): vedi il prompt log, riga 3.

****Dichiarazione d'uso dell'IA.**** Gemini (Gem della classe) per la prima versione del codice e per una spiegazione.

Fatti da me: specifica, test, correzione dello sconto, scelta di lasciare il test rosso.

Rubrica da compilare

Criterio	Peso	Marco: livello ed evidenza	Giulia: livello ed evidenza
Specifica	20%		
Test prima del codice	20%		
Spiegazione orale e modifica dal vivo	40%	<i>(si valuta all'orale)</i>	<i>(si valuta all'orale)</i>
Processo: prompt log, commit, PR	20%		
Il programma funziona	requisito		

Livelli: insufficiente · sufficiente · buono · ottimo (descrittori nella rubrica di `report-fine-giornata.md`).

Tre domande per l'orale di Marco 1. 2. 3.

Tre domande per l'orale di Giulia 1. 2. 3.